DHAR

Bare Metal. Absolute Control.

VOLUME III — ECOSYSTEM, TRUST & ADOPTION

TOOLING DHAR

"A tool that cannot explain its own refusal has not finished its job."

DOCUMENT	Tooling Dhar — Developer Tools, Diagnostics & the Playground
VERSION	1.0
DATE	October 26, 2026
STATUS	Proposed
PART OF	The Canon of Dhar, Volume III
DEPENDS ON	Stage-delta §5, Stage-epsilon §3.1, Amendments A-1 and A-2
AUDIENCE	Tooling engineers, early adopters

Contents

1. Mandate 3

2. Diagnostics and Error Messages 3

3. Editor Support and the Language Server 4

4. Commands and the Documentation Generator 4

4.1 Commands 4

4.2 The documentation generator 4

5. The In-Browser Playground 5

5.1 What it is 5

5.2 What a visitor can do 5

5.3 The first version: a compile service 5

5.4 The later version: the compiler in the page 5

5.5 What it never does 6

5.6 What it must meet 6

6. Acceptance Criteria 6

7. Relationship to Other Volumes 6

1. Mandate

Make writing, reading, checking and trying Dhar code easy enough that the language's strict rules help a person instead of stopping them. Stage-delta §5 lists the tools in the standard distribution. This document states what each of them must do, how the compiler speaks to the person using it, and what the in-browser playground is.

Dhar is a language for trustworthy computation: it goes first to cybersecurity, then to artificial intelligence, then to quantum computing, with every other domain kept as modules (Spec0 §5.1, §5.2; Amendment A-1). For its tools that means a refusal has to be explained as carefully as a result is reported, because a language that rejects insecure code is only as useful as the message it rejects it with. The tools are meant for learners on modest machines as much as for engineers (Casting Dhar §3.2).

2. Diagnostics and Error Messages

A diagnostic is a message from the compiler or from a tool that tells a person that something is wrong or worth noticing. Every diagnostic obeys these rules:

1. **Where.** It names the file, line and column, and shows the line of source with the offending part marked.
2. **What and why.** It states in words which rule was broken, and names the rule with a stable identifier, such as those named in Spec1 §14.5. An identifier does not change between releases, so that a search for it finds every discussion of it.
3. **How to fix.** Where one fix is clear it says so; where there are several it names the most likely.
4. **The same on every target.** One program gives the same diagnostic on every committed target (Amendment A-2 §3.1). A diagnostic about a construct that a target cannot express names the target and the reason (Stage-alpha §2.2, §3.1).
5. **A refusal is complete.** A refusal, whether of an unsupported target or of a construct outside the bootstrap subset, ends with a non-zero exit status and no output file (Amendment A-2 §2; Stage-alpha §2.2).
6. **A run-time failure is defined.** A failed bounds check writes the one defined message and stops with the one defined status on every target (Spec1 §14.4; Stage-alpha §3.3).
7. **Readable without colour.** The meaning of a message never depends on colour, and a message reads correctly in a plain text file. Each diagnostic is also available in a machine-readable form, one object per diagnostic, for editors and for continuous integration.
8. **No flood.** One cause does not produce a hundred follow-on messages. The number shown has a stated default that a flag changes.
9. **A secret is never printed.** A diagnostic, a failure message or a crash report never prints the contents of a secret value or of a secret buffer (Spec1 §14.5). It names the binding instead.

3. Editor Support and the Language Server

Dhar provides a language server that speaks the Language Server Protocol (LSP), so that any editor that supports the protocol works with Dhar without a plug-in of its own.

1. **The minimum** is diagnostics and go-to-definition (Stage-delta §5, §7).
2. **Beyond the minimum**, in this order: hover that shows the type of a binding, inferred types included (Spec1 §4.3), and shows the secret qualifier where there is one (Spec1 §14.5); completion; find references; rename; formatting through `dhar fmt`; and the list of symbols in a file.
3. **One front end.** The server uses the compiler's own lexer, parser and type checker, never a second one, so that the editor and the compiler cannot disagree about a program and the diagnostics of §2 are the same in both.
4. **Local.** The server works with no network, and it never sends source code anywhere.
5. **Highlighting.** A syntax-highlighting definition is published, so that highlighting does not depend on the server running.

4. Commands and the Documentation Generator

4.1 Commands

Stage-delta §5 names `dhar build`, `dhar test` and `dhar fmt`. These further commands are required:

- `dhar build` names its target on the command line, and refuses any target outside the committed list with the refusal of §2, rule 5 (Amendment A-2 §3.1).
- `dhar test --vectors` runs the published known-answer vectors of `std.crypto`, post-quantum algorithms included (gates S3 and Q1, Amendment A-1 §9). It reports each vector set by name and by the standard it comes from, and exits non-zero if any fails. The vectors are data kept apart from code, and outside implementations are used only to produce and check them, never as dependencies (Stage-delta §3.8).
- `dhar verify-build` takes a source tree and a release binary, rebuilds, and compares the two byte for byte. It reports either identical, or the first offset at which they differ, and it checks the release's signature and bill of materials (Stage-epsilon §2, §3.2; Amendment A-1, S4). It works with no network.
- `dhar sbom` writes the software bill of materials of a build from Kun's resolution records, with oracle dependencies listed apart from dependencies (Stage-delta §3.7, §3.8).

4.2 The documentation generator

The generator produces reference documentation from the source. A run of `#` comment lines immediately above a task, mold, alloy, bond or exposed item documents it. This is a convention of the tool and adds nothing to the grammar (Spec1 §2.3). The first line is a summary; the rest is free text.

1. **Examples that run.** An example in a documentation comment is compiled and run by `dhar test`, and a failing example fails the build, so that documentation cannot drift from the code.
2. **Signatures are complete.** A signature shows its types, inferred types included, and the secret qualifier wherever a value is secret.
3. **Output.** Static pages and plain text, both readable offline.

5. The In-Browser Playground

5.1 What it is

A page on Dhar's website where anyone can write a Dhar program, run it and see its output, with nothing to install and no account. It is for learning, for sharing a small example, and for trying Dhar before deciding to install it (Casting Dhar §2.5). It requires the WebAssembly target at gate G1 (Amendment A-2 §7), and the page does not open to the public before that gate has passed.

5.2 What a visitor can do

1. Edit a program, run it, and see what it writes to standard output and standard error, including the defined failure message of §2, rule 6.
2. Start from examples: one for each of the three tracks, a first program, and a program that fails a bounds check.
3. Share a program by a link. The program is held in the fragment of the address, which a browser does not send to a server.
4. Learn which compiler built it: the page shows the version of the compiler it used, so that a result can be reproduced.

The playground accepts exactly what the compiler of that version accepts for the WebAssembly target. A program that uses a call the target cannot offer, such as a file when no directory is granted, is refused with the diagnostic of §2 (Amendment A-2 §5.1).

5.3 The first version: a compile service

The page sends the source to the compile service (Stage-epsilon §3.1), which compiles it in a sandbox for the WebAssembly target and returns the module. The page runs the module in the browser's own WebAssembly sandbox, in a worker that the page can end at any time, with a small script of Dhar's own that provides only the imports of the committed target (Amendment A-2 §3.1):

- `fd_write` on descriptors 1 and 2 appends to the output panel;
- `fd_read` on descriptor 0 reads from an input box, or reports the end of input;
- `proc_exit` ends the run and shows the exit status.

Every other import is absent, so a module that needs one cannot start. The page stops a run that exceeds a stated time, limits the amount of output, and refuses a module that declares more memory than a stated size. The service never runs the program (Stage-epsilon §3.1).

5.4 The later version: the compiler in the page

When Stage-beta's compiler targets WebAssembly, the compiler itself runs in the page, and compiling and running both happen in the visitor's browser. The compile service is then no longer needed for compiling (Stage-epsilon §3.1, item 5). The script provides an in-memory file area for the compiler's input and output, never the visitor's own disk, and the page aims to need a connection only to be loaded.

5.5 What it never does

1. It has no accounts, and it keeps no source code, no output and no identity of a visitor on any server (Stage-epsilon §3.1; The Hallmark of Dhar §7.7).
2. A running program has no network and no access to the visitor's files.
3. A program shared by a link is held only in the link. Anyone given the link can read the program, and the page says so.
4. It makes no statement about the security of a program run in it. A statement about the playground names the test that backs it (Tempering Dhar §11).

5.6 What it must meet

The service meets Stage-epsilon §3.1 and the tests of Stage-epsilon §4. Its terms are The Hallmark of Dhar §8 and the data it may hold is §7.7 of the same document. The script that provides the imports is Dhar's own code, small enough to read, published and tested: the conformance battery (Stage-alpha §3.3) runs through it in a browser and passes. A program the playground accepts compiles unchanged for every other committed target with the same observable behaviour (Stage-epsilon §4, criterion 9).

6. Acceptance Criteria

1. Every named diagnostic of Spec1 §14.5 and every refusal of Stage-alpha has a test that checks the location, the identifier, the exit status and that no output file exists.
2. The same program gives the same diagnostic on every committed target on which it can be compiled.
3. The language server is verified against at least one real editor for diagnostics, go-to-definition and hover on inferred types.
4. `dhar test --vectors`, `dhar verify-build` and `dhar sbom` work with no network. `dhar verify-build` detects a change of one byte, and `dhar sbom` lists oracle dependencies apart.
5. The documentation generator builds the reference for the standard library, and a deliberately wrong example fails the build.
6. A program with a secret in scope that fails at run time prints neither the secret nor its contents (§2, rule 9).
7. The conformance battery passes in a browser through the script of §5.3; a run that never ends is stopped within the stated time; a program shared by link is restored exactly.
8. The tests of Stage-epsilon §4, criteria 8 and 9, pass for the compile service.

7. Relationship to Other Volumes

THIS DOCUMENT DEFINES	RATIFIED IN
The messages that report Spec1's rules and bounds failures	Spec1 §14.4, §14.5; Stage-alpha §2.2, §3.3
The commands it extends	Stage-delta §5
The service behind the playground, and its limits	Stage-epsilon §3.1

THIS DOCUMENT DEFINES	RATIFIED IN
The WebAssembly target and its imports	Amendment A-2 §3.1, §5.1, §7
The terms and the data of the playground	The Hallmark of Dhar §7.7, §8
The use of the playground in teaching	Casting Dhar §2.5

Revision log

Tooling Dhar — Developer Tools, Diagnostics & the Playground · v1.0 · October 26, 2026 · Proposed. Names two outside interfaces: the WebAssembly System Interface (WASI), preview 1, and the Language Server Protocol (LSP). Complete and open for revision before ratification (Master Index §3.2).

VERSION	DATE	CHANGE
v1.0	October 26, 2026	First edition, written under Amendments A-1 and A-2: diagnostics, the language server, the commands and the documentation generator, and the in-browser playground (§5). Proposed for ratification by the project lead.